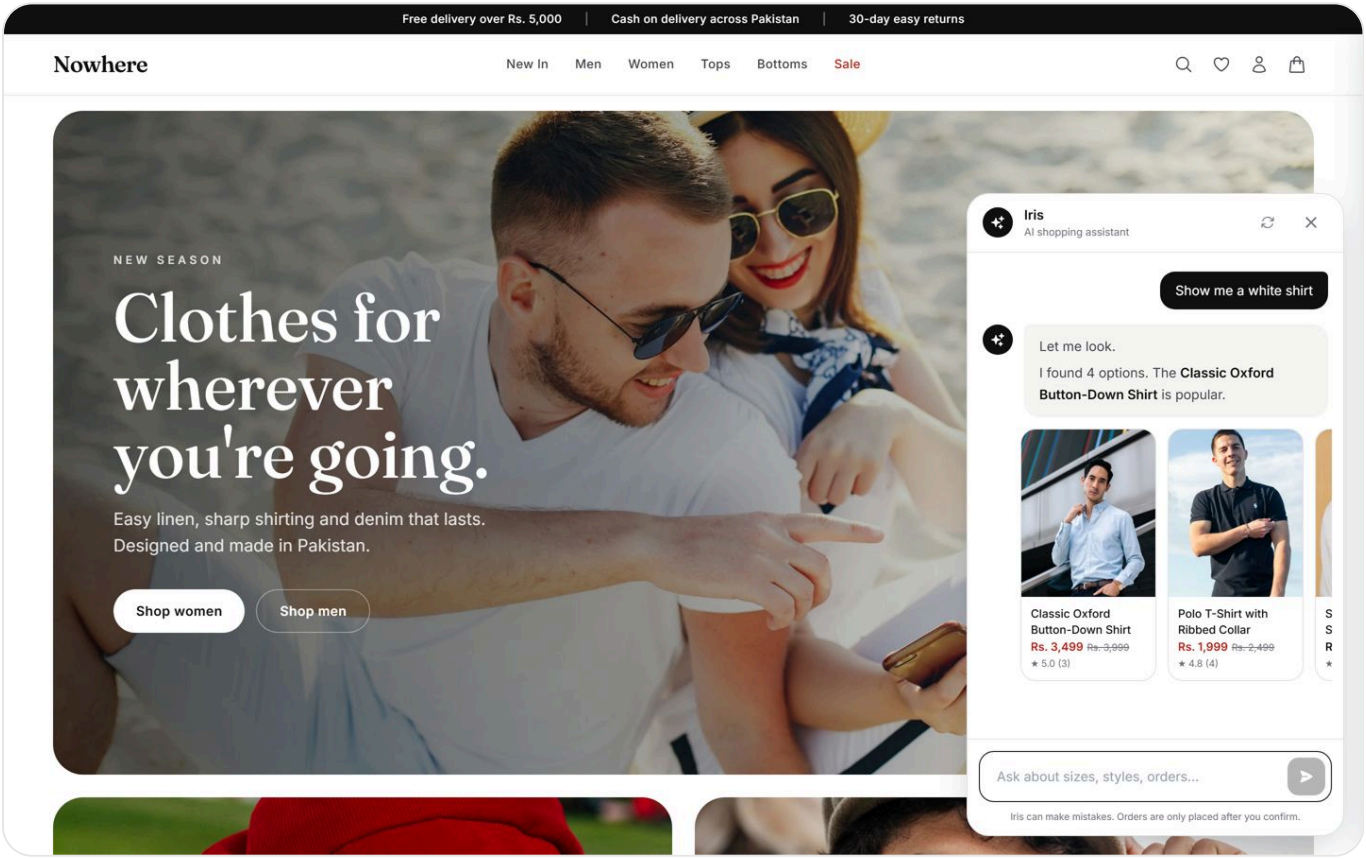
Nowhere

PROJECT OVERVIEW · OCTOBER 2026

A MERN clothing store with an AI assistant that shops for you.

Storefront, guest checkout, order tracking, admin dashboard and **Iris**, a free-LLM shopping agent that searches, fills the bag and places cash-on-delivery orders after the shopper confirms.



Syed Ibrahim Ali
Full-Stack & AI Engineer

Stack
MongoDB · Express · React · Node

AI
Groq / Google Gemini (free tiers)

Repository
github.com/ibrahim123-sia/Nowhere

01 · SUMMARY

What Nowhere is

Nowhere is a production-style online store for a Pakistani clothing brand selling men's and women's top wear and bottom wear. It is a single-page React application backed by an Express REST API and MongoDB. Prices are in PKR, payment is cash on delivery, and shoppers can buy with or without an account.

This release was a full audit and rebuild: security holes and functional bugs were fixed, the catalogue data was upgraded with real photography and product detail, the interface was redesigned around an editorial fashion look, the features customers expect from leading stores were added, and an AI shopping agent was built into the site.

40 PRODUCTS, 80 REAL PHOTOS	192 SAMPLE REVIEWS WITH FIT NOTES	13 AGENT TOOLS (SEARCH TO ORDER)
30+ BUGS AND SECURITY ISSUES FIXED	86 AUTOMATED API & AGENT CHECKS	0 KNOWN PROD DEPENDENCY VULNS

Who it serves

Guests

Browse, filter, review sizes, add to bag and check out with only an email. Track orders with the order number and email. Chat with Iris.

Customers

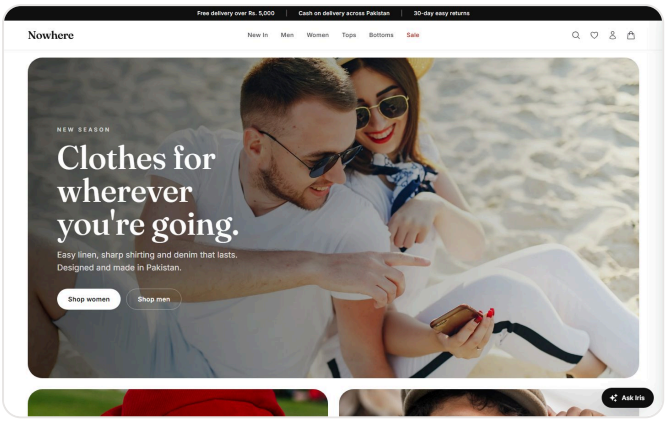
Everything guests can do, plus saved address, order history, cancel and "buy again", reviews, and a cart that follows them across devices.

Admins

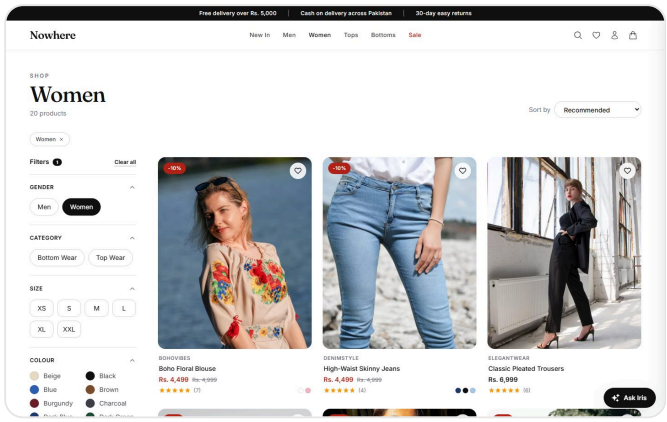
Sales dashboard, low-stock alerts, product catalogue management, order fulfilment workflow and user management.

02 · THE PRODUCT

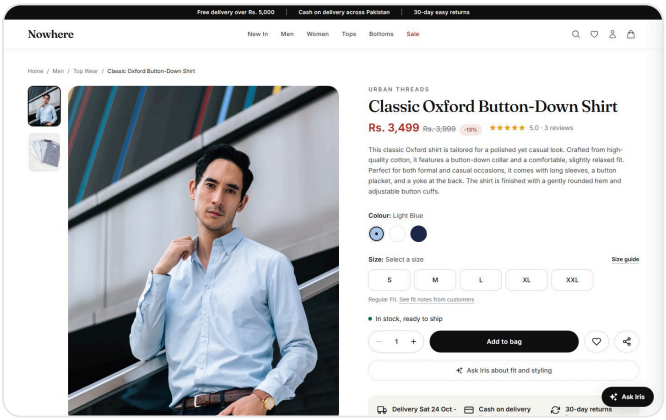
Key screens



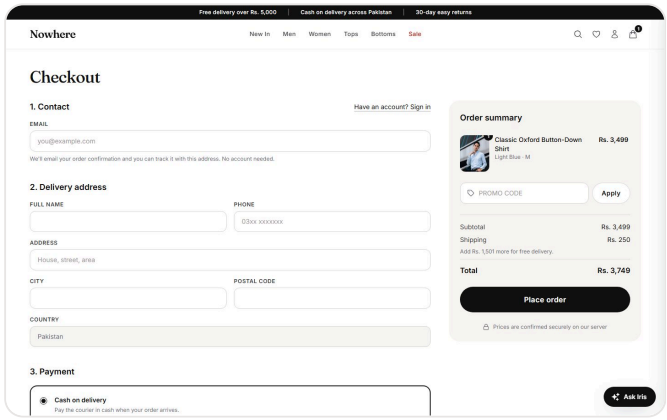
Home: editorial hero, category tiles and product carousels.



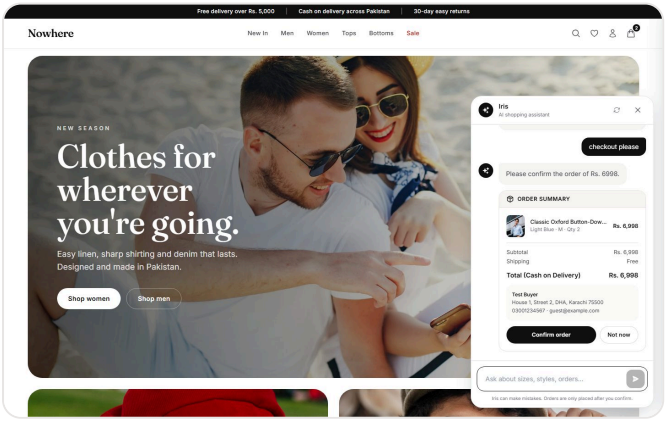
Collection: filters generated from the catalogue, chips, sorting.



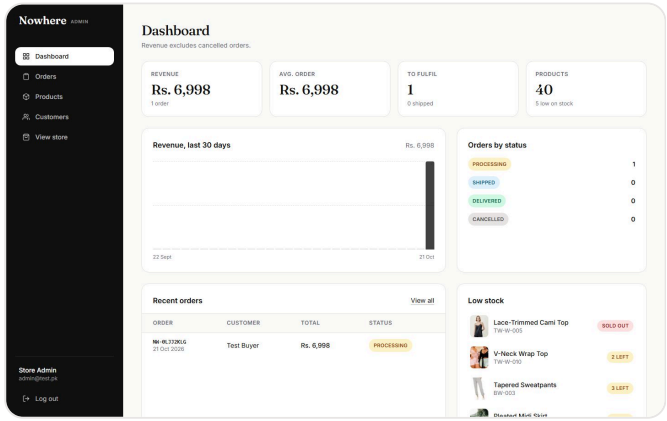
Product: gallery, sale price, named colours, size guide, stock, delivery estimate.



Checkout: guest email, saved address, promo codes, server-priced totals, COD.



Iris prepares an order summary; nothing is placed until the shopper confirms.



Admin dashboard: revenue, orders by status, 30-day sales, low stock.

03 · FEATURES

Feature set

Shopping

- Home with new arrivals, best sellers, sale and category carousels
- Filters for gender, category, size, colour, material, brand, price, stock and sale, built from live catalogue facets
- Live search suggestions; sorting by newest, rating and price
- Product pages with sale %, named colour swatches, size guide, stock status, delivery window, fabric & care, share
- Reviews with rating breakdown, fit meter and verified-buyer badges
- Persistent wishlist and recently viewed products

Checkout & orders

- Guest checkout with email; optional accounts
- Cash on delivery, Rs. 250 shipping, free over Rs. 5,000
- Promo codes `WELCOME10` (first order) and `NOWHERE500`
- Atomic stock reservation; stock returns on cancellation
- Order numbers, status timeline, cancel while processing, buy again
- Guest order tracking with order number + email; confirmation emails

Iris, the AI shopping agent

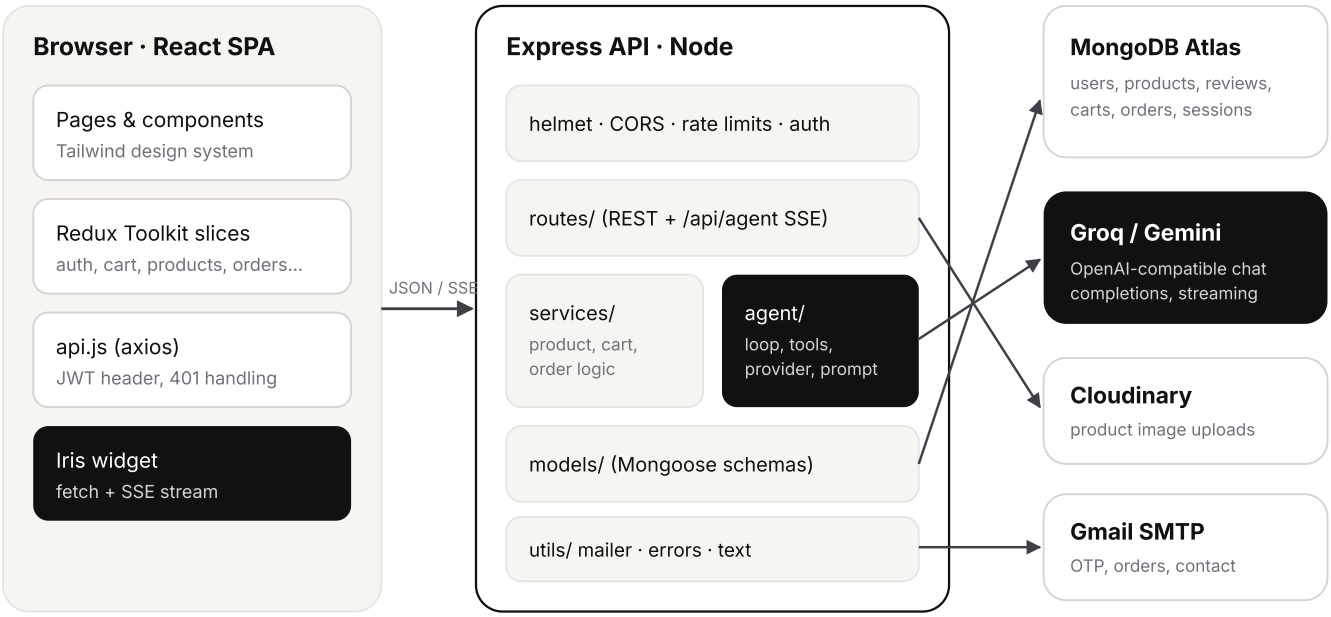
- Chat on every page, streamed replies, English / Urdu / Roman Urdu
- Searches, explains fit and fabric, manages the cart, checks policies
- Prepares and places COD orders, tracks, cancels and re-orders
- Product cards with Add to bag and Buy now, a live bag card, order summary and order-placed cards
- Two-step confirmation enforced by the server
- Runs free on Groq or Gemini with automatic fallback

Accounts & admin

- JWT auth, email OTP verification, password reset, profile, saved address, password change
- Dashboard: revenue, AOV, status counts, 30-day chart, recent orders, low stock
- Single product form for add/edit with every field and image management
- Order workflow (ship, deliver, cancel) with history; user roles

04 · ARCHITECTURE

How the system fits together



Routes stay thin; business rules live in services so the REST API and the agent's tools share exactly the same validation, pricing and stock logic.

Design decisions

Decision	Why
Service layer (`services/`)	The agent must obey the same rules as the website. One implementation of pricing, stock and ownership avoids two code paths drifting apart.
Server-decided identity	The cart and order owner comes from the JWT or the guest id, never from the request body or the model's tool arguments.
Guest id in local storage	Guests get a persistent cart, chat and order history on their device without an account; it merges into the account on sign-in.
OpenAI-compatible provider layer	Groq and Gemini both expose this format, so a single `fetch` + SSE parser supports both free providers with fallback, and no vendor SDK is required.
Two-step order confirmation	Language models can misread intent. Placing or cancelling an order needs a prepare step and a confirmation in a later message, enforced server-side.
Aggregation-based search	Sorting and price filters use the effective (sale) price, and facets are computed from the published catalogue.
SEO for a single-page app	Link previews don't run JavaScript, so a serverless function returns product pages with title, image, price and schema.org data in the HTML. In the app, every page sets its own meta tags; products have readable URLs; a live sitemap lists the catalogue.

Technology and data model

Layer	Technologies
Frontend	React 18, Vite 6, Redux Toolkit, React Router 7, Axios, Tailwind CSS 3, React Icons, Sonner
Backend	Node.js 20+, Express 4, Mongoose 8, jsonwebtoken 9, bcryptjs, Nodemailer, Multer + Cloudinary, express-rate-limit, helmet, CORS
AI	Groq `openai/gpt-oss-120b` (default) or Gemini `gemini-3.8-flash`, or any OpenAI-compatible endpoint; configurable per environment
Data	MongoDB Atlas
Hosting	Vercel (static frontend + Node serverless API)
Quality	ESLint; end-to-end API and agent tests (`npm test`) using mongodb-memory-server and a mock LLM; GitHub Actions CI runs tests, lint and a production build on every push

Collections

Collection	Key fields
users	name, email (unique, lowercase), password (hashed, never returned), role, phone, shippingAddress, hashed OTP + expiry + attempts, isVerified
products	name, description, price, discountPrice, countInStock, sku, category, gender, collections, brand, material, fit, season, occasion[], highlights[], careInstructions[], sizes[], colors[], images[], isPublished, isFeatured, rating, numReviews
reviews new	product, user (optional for samples), name, rating, title, comment, fit, sizePurchased, verifiedPurchase; one per user per product
carts	user *or* guestId, products[{productId, name, image, price, size, color, quantity}], totalPrice
orders	orderNumber, user *or* guestId + email, orderItems[], shippingAddress (name, phone…), itemsPrice, shippingPrice, discount, couponCode, totalPrice, paymentMethod, isPaid, status, statusHistory[], cancelledAt, source (web / assistant)
agentsessions new	user or guestId, messages[] (model history), transcript[] (for the chat window), turn, pendingAction, lastActiveAt (7-day TTL)
subscribers	email, subscribedAt

API reference

Base URL `/api`. "Guest or user" routes accept an optional JWT and otherwise identify the shopper by `guestId`.

Endpoint	Access	Purpose
`POST /users/register · verify-otp · resend-otp`	Public	Sign up with email OTP
`POST /users/login · forgot-password · reset-password`	Public	Sign in and password reset
`GET · PUT /users/profile`	User	Profile, saved address, password change
`GET /products`	Public	Search, filters, sorting, paging
`GET /products/facets · suggest · best-sellers · new-arrivals · sitemap`	Public	Filter values, search suggestions, lists
`GET /products/:id · /similar/:id`	Public	Product detail and similar items
`GET · POST /products/:id/reviews`	Public · User	Reviews with summary; write a review
`POST · PUT · DELETE /products`	Admin	Manage the catalogue
`GET · POST · PUT · DELETE /cart`	Guest or user	Cart with stock, size and colour validation
`POST /cart/merge · /cart/reorder`	User	Merge guest cart; re-add a past order
`POST /checkout/quote · POST /checkout`	Guest or user	Price the cart; place a COD order
`GET /orders/my-orders · /orders/:id`	User (owner/admin)	Order history and detail
`PUT /orders/:id/cancel · POST /orders/track`	User · Public	Cancel; guest tracking by number + email
`POST /agent/chat`	Guest or user	Send a message; reply streams as SSE
`POST /agent/action · GET/DELETE /agent/session/:id · GET /agent/status`	Guest or user	Confirm/cancel buttons, history, reset, availability
`GET /store/config · POST /subscribe · POST /contact`	Public	Store settings, newsletter, contact form
`/admin/users · /admin/products · /admin/orders(/stats)`	Admin	User, catalogue and order management, dashboard
`POST /upload`	Admin	Image upload to Cloudinary (5 MB, images only)

07 · AUDIT

What the audit found and fixed

Security

Issue	Fix
Cart API trusted a `userId` from the request body, so anyone could edit anyone's cart	Owner is resolved from the JWT or guest id only
Checkout took item prices and totals from the browser	All prices, shipping and discounts computed on the server
Any signed-in user could read any order by id	Owner or admin only; same 404 for "missing" and "not yours"
Image upload endpoint was public	Admin only, 5 MB, image types only
Password hashes returned by `/profile` and the admin user list	Password excluded at the schema level
Public `/test-email` endpoint; tokens logged to the console	Removed
OTPs stored in plain text with unlimited guesses; rate limit skipped successful requests	Hashed OTPs, constant-time compare, attempt limits, proper rate limits
Unescaped user input used as a RegExp; open CORS; no security headers	Escaped search, CORS allowlist, helmet, body size limit
Vulnerable jsonwebtoken 8, nodemailer and express-rate-limit	Upgraded; `npm audit` clean for production

Functional bugs

- Adding a product from the admin failed (missing category/collection); editing navigated away before saving; the admin slice called routes that didn't exist.
- Seeded and admin-created accounts were unverified and could not sign in.
- Cash-on-delivery orders were marked as paid at checkout; stock never decreased; shipping name and phone were dropped.
- Discount prices were stored but never charged or shown; ratings showed "0 reviews".
- Hero and footer links pointed at collections that didn't exist; the home "Top Wears" section loaded bottom wear; filter lists were hard-coded and misspelt.
- Wishlist and share buttons did nothing; the contact form faked success; FAQ promised card payments and international shipping.
- Product images were random stock photos (e.g. a city skyline for a shirt) and prices showed as "Rs. 40".
- The navbar wasn't actually sticky; pages didn't scroll to top; there was no 404 page.

08 · RESEARCH

How Nowhere compares

Benchmarks were leading fashion and marketplace stores and published e-commerce UX research. Apparel shoppers rely heavily on sizing information and fit feedback in reviews, expect shipping and returns to be visible before they commit, and increasingly use AI assistants that can act (for example adding items to the cart for review before checkout).

Capability	Before	Now
Product photography	Random photos	Two matching on-model / flat-lay photos per item, hover swap
Sizing help	None	Size guide, fit description, fit meter from reviews
Reviews	Count field only	Full reviews, breakdown, verified buyers, write a review
Filters	Hard-coded lists	Live facets, chips, price presets, stock and sale
Search	Submit only	Live suggestions and popular searches
Guest checkout	Forced registration	Email-only checkout and tracking
Order tracking	Basic status	Order numbers, timeline, cancel, buy again, emails
Promotions	"10% off" text with no code	Working promo codes priced on the server
Shipping & returns visibility	Generic banner	On product, cart, checkout and FAQ, with delivery dates
Wishlist	Visual only	Persistent wishlist page
AI assistant	None	Agent that searches, fills the bag, orders, tracks and cancels
Admin insight	Three counters	Revenue, AOV, status, 30-day chart, low stock

Recommended next steps

- Online payments (JazzCash, Easypaisa or a card gateway) alongside COD.
- Stock per size and colour instead of per product.
- Return and exchange requests from the order page (and through Iris).
- Customer photos in reviews; email or WhatsApp shipping notifications.
- Analytics on searches and assistant conversations to improve the catalogue.

09 · OPERATIONS

Setup and deployment

Environment

Variable	Meaning
`MONGODB_URI`, `JWT_SECRET`	Database and token signing secret
`CLIENT_URL`	Allowed frontend origin(s), comma separated (production)
`CLOUDINARY_*`, `EMAIL_USER`, `EMAIL_PASS`	Image uploads and Gmail app password
`AGENT_PROVIDER`	`groq` (default), `gemini` or `custom`
`GROQ_API_KEY`, `GROQ_MODEL`	Free key from console.groq.com; model override
`GEMINI_API_KEY`, `GEMINI_MODEL`	Free key from aistudio.google.com; also used as fallback
`AGENT_BASE_URL`, `AGENT_API_KEY`, `AGENT_MODEL`	Any other OpenAI-compatible endpoint
`ADMIN_NAME`, `ADMIN_EMAIL`, `ADMIN_PASSWORD`	Admin created by `npm run seed`

Scripts

Command	What it does
`npm run dev` (backend / frontend)	API with nodemon on :9000; Vite on :5173
`npm run seed`	Fresh database: sample catalogue and reviews, upserts a verified admin. Never deletes users or orders.
`npm run enhance [-- --dry]`	Upgrades an existing catalogue in place by SKU, backfills order numbers, verifies admins.
`npm run build` / `npm run lint`	Production build and ESLint (frontend)
`npm test`	92 end-to-end API and assistant checks against an in-memory database and a mock LLM

> **DEPLOYMENT CHECKLIST**
>
> Add `GROQ_API_KEY` (and optionally `GEMINI_API_KEY`) and `CLIENT_URL` to the backend's Vercel environment and `VITE_BACKEND_URL` to the frontend's (plus `SITE_URL` on a custom domain), redeploy, then open the site and send Iris a message. If no key is set the store works normally and the chat shows that the assistant is offline. Change the seeded admin password after the first sign-in.